

Does the Global South Receive Less Guardian Coverage as Crises Get More Serious?

Table of Contents

1. Introduction
2. Data Acquisition
3. Preparation for Data Analysis
4. Exploratory Data Analysis
5. Conclusion

1. Introduction

The past decade has seen a succession of major conflict outbreaks, from the Syrian civil war and the Tigray conflict in Ethiopia, to the Russian invasion of Ukraine in 2022 and the escalation of violence in Sudan and Gaza in 2023. These crises have tested how global media allocates attention, and whether that allocation reflects the scale of human suffering or something else entirely.

Media coverage of global crises shapes public awareness and, in turn, political and humanitarian response. If a newspaper's coverage is disproportionately concentrated on Global North conflicts, regardless of their relative severity, this has real implications for how its readership understands and prioritises global suffering. The Guardian is a particularly meaningful case study: it has an explicit editorial commitment to international reporting and positions itself as a progressive outlet with global reach.

The concept of proximity bias in journalism is well-established in media studies literature: outlets tend to devote more coverage to geographically or culturally proximate events. However, existing work is largely qualitative or limited to specific conflicts. No publicly available analysis applies a severity-controlled, data-driven framework to Guardian coverage across a decade-long window. The originality of this project lies in using ACLED fatality data as a quantitative severity benchmark, allowing us to ask not just "who gets covered?" but "who gets covered relative to how serious their crisis is?"

Research Questions

1. Is there a measurable disparity in Guardian coverage volume between Global South and Global North countries, controlling for conflict severity?

2. Do structural features of articles, length and section placement, differ systematically by region?
 3. Does The Guardian respond to casualty spikes differently depending on region, and has any disparity narrowed or widened over time (2015–2024)?
-

2. Data Acquisition

We use three main data sources:

- **Guardian API** (primary): Article metadata for all countries with recorded conflict, 2015–2024
- **ACLED** (supplementary): Reported fatalities by country-year, used as our conflict severity proxy — [Link here](#)
- **Our World in Data / World Bank** (supplementary): Country income classification, used to assign Global South / Global North labels — [Link here](#)

We first load the necessary libraries and store each group member's Guardian API key in a `keys.json` file, which is excluded from the repository via `.gitignore` to avoid exposing credentials. A `keys_template.json` is provided — copy it, rename to `keys.json`, and fill in your key before running.

```
In [70]: import requests
import pandas as pd
import numpy as np
import json
import time
import os
from pathlib import Path
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio
from scipy import stats
from scipy.stats import pearsonr, levene
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.graphics.gofplots import qqplot
pio.renderers.default = "png"
with open("../keys.json") as f:
    keys = json.load(f)
API_KEYS = [
    keys["guardian"]["api_key_1"],
    keys["guardian"]["api_key_2"],
    keys["guardian"]["api_key_3"]
]
```

2.1 ACLED and World Bank Data Collection

We first download the World Bank income classification from Our World in Data and load the ACLED fatalities file as pandas dataframes, named `work_bank_df` and `fatalities` respectively. The raw ACLED file (`number_of_reported_fatalities_by_country-year_as-of-03Apr2026.csv`) must be present in the `/data` folder before running this section — it is included in the repository.

We then combine the two datasets by assigning each country a binary label based on its most recent World Bank classification: **Global North** (high income) or **Global South** (all others). Manual overrides are applied for geopolitically misclassified countries — Gulf states are reclassified as Global South despite their high income, and Ukraine as Global North. This cleaned classification is saved to `worldbank_income_clean.csv`.

Before merging, we apply a name harmonisation step to resolve inconsistencies between the two sources (for example, the World Bank lists 'Czechia' while ACLED uses 'Czech Republic'). We also filter ACLED to our study window of 2015–2024, excluding sparse pre-2015 coverage and future years beyond our Guardian date range.

The resulting dataset, with region labels attached, is saved to `cleaned_fatalities.csv` in the `/data` folder.

```
In [71]: data_path = Path.cwd().parent / "data"
work_bank_df = pd.read_csv(
    "https://ourworldindata.org/grapher/world-bank-income-groups.csv"
    "?v=1&csvType=full&useColumnShortNames=false",
    storage_options={"User-Agent": "Our World In Data data fetch/1.0"}
)
work_bank_df.to_csv(f"{data_path}/world-bank-income-groups.csv", index=False)
WB_TO_ACLED = {
    "Congo": "Republic of Congo",
    "Czechia": "Czech Republic",
    "Eswatini": "eSwatini",
    "Congo, Dem. Rep.": "Democratic Republic of Congo",
    "Côte d'Ivoire": "Ivory Coast",
    "Korea, Rep.": "South Korea",
    "Korea, Dem. People's Rep.": "North Korea",
    "Lao PDR": "Laos",
    "Syrian Arab Republic": "Syria",
    "Yemen, Rep.": "Yemen",
    "Venezuela, RB": "Venezuela",
    "Iran, Islamic Rep.": "Iran",
    "Egypt, Arab Rep.": "Egypt",
    "Kyrgyz Republic": "Kyrgyzstan",
    "Turkiye": "Turkey",
}
work_bank_df["Entity"] = work_bank_df["Entity"].replace(WB_TO_ACLED)
INCOME_COL = "World Bank's income classification"
wb_latest = (
    work_bank_df
    .sort_values("Year", ascending=False)
    .drop_duplicates(subset="Entity")
    .rename(columns={"Entity": "country", "Year": "wb_year"})
)
wb_latest["region_group"] = wb_latest[INCOME_COL].apply(
    lambda x: "Global North" if x == "High-income countries" else "Global South"
)
for c in ["Ukraine"]:
```

```
wb_latest.loc[wb_latest["country"] == c, "region_group"] = "Global North"
for c in ["Saudi Arabia", "Qatar", "Kuwait", "Bahrain", "Oman", "United Arab Emirates"]:
    wb_latest.loc[wb_latest["country"] == c, "region_group"] = "Global South"
wb_latest.to_csv(f"{data_path}/worldbank_income_clean.csv", index=False)
print(f"Saved worldbank_income_clean.csv: {wb_latest.shape}")

global_north = wb_latest[wb_latest["region_group"] == "Global North"]["country"]
global_south = wb_latest[wb_latest["region_group"] == "Global South"]["country"]
fatalities = pd.read_csv(
    f"{data_path}/number_of_reported_fatalities_by_country-year-as-of-03April-2020.csv",
    skiprows=1
)

fatalities = fatalities.loc[:, ~fatalities.columns.str.contains('^Unnamed')]
fatalities = fatalities[(fatalities['YEAR'] >= 2015) & (fatalities['YEAR'] < 2020)]
fatalities["region_group"] = fatalities["COUNTRY"].apply(
    lambda x: "Global North" if x in global_north else "Global South"
)

fatalities.loc[fatalities["COUNTRY"] == "Ivory Coast", "region_group"] = "Global North"
fatalities.to_csv(f"{data_path}/cleaned_fatalities.csv", index=False)
acled = pd.read_csv(f"{data_path}/cleaned_fatalities.csv")
print(f"\nSaved cleaned_fatalities.csv: {acled.shape}")
print(acled['region_group'].value_counts())
print(acled.head(2))
```

Saved worldbank_income_clean.csv: (226, 5)

Saved cleaned_fatalities.csv: (1634, 4)

region_group

Global South	1221
--------------	------

Global North	413
--------------	-----

```
Name: count, dtype: int64
```

COUNTRY	YEAR	FATALITIES	region_group
---------	------	------------	--------------

Country	Year	Population	Region
Afghanistan	2017	36360	Global South

1	Afghanistan	2018	42991	Global South
---	-------------	------	-------	--------------

The cleaned ACLED dataset contains 1,634 country-year observations across four columns. Of these, 1,221 belong to Global South countries and 413 to Global North — a reasonable split that reflects the greater number of conflict-affected countries in the Global South. The dataset is now ready to merge with the Guardian article data.

2.2 Guardian API Data Collection

Once the necessary libraries and API keys are loaded, we query the Guardian's search endpoint for every country present in the ACLED dataset. Country names are converted to Guardian tag slugs programmatically, with a manual corrections dictionary for edge cases. USA, UK, and Australia are queried by section rather than tag, as they have dedicated top-level sections on the Guardian.

For each article we collect: publication date, section name, word count, article title, editorial pillar, and author byline.

Step 1: Load the ACLED file and convert country names into Guardian API-compatible tag slugs.

```
In [72]: acled = pd.read_csv("../data/cleaned_fatalities.csv")
FROM_DATE = "2015-01-01"
TO_DATE    = "2024-12-31"
acled_countries = acled["COUNTRY"].unique().tolist()
def name_to_slug(name):
```

```

    return name.lower().strip().replace(" ", "-").replace("'", "").replace(
auto_slugs = {country: name_to_slug(country) for country in acled_countries}
SLUG_CORRECTIONS = {
    "Democratic Republic of Congo": "democratic-republic-of-congo",
    "Republic of Congo": "republic-of-congo",
    "United States": "us-news",
    "Myanmar": "myanmar",
    "Ivory Coast": "ivory-coast",
    "Burkina Faso": "burkina-faso",
    "South Sudan": "south-sudan",
    "Central African Republic": "central-african-republic",
    "Bosnia-Herzegovina": "bosnia-and-herzegovina",
    "North Korea": "north-korea",
    "South Korea": "south-korea",
    "Saudi Arabia": "saudi-arabia",
    "Sri Lanka": "sri-lanka",
    "Papua New Guinea": "papua-new-guinea",
}
COUNTRY_TO_SLUG = {**auto_slugs, **SLUG_CORRECTIONS}
COUNTRY_LIST = [(country, COUNTRY_TO_SLUG[country]) for country in acled_countries]
print(f"\nTotal countries to query: {len(COUNTRY_LIST)}")
print("\nSample mappings:")
for name, slug in COUNTRY_LIST[:5]:
    print(f"    {name} -> world/{slug}")

```

Total countries to query: 245

Sample mappings:

```

Afghanistan -> world/afghanistan
Akrotiri and Dhekelia -> world/akrotiri-and-dhekelia
Albania -> world/albania
Algeria -> world/algeria
American Samoa -> world/american-samoa

```

Step 2: We define two collection functions: a standard version for most countries (fetch_articles), and a slower version with longer delays and timeout handling (fetch_articles_slow) for high-volume sections such as USA, UK, and Australia, where the standard function risks stalling during collection.

```

In [73]: BASE_URL = "https://content.guardianapis.com/search"
current_key = 0
def _build_params(query_param, page):
    return {
        **query_param,
        "from-date": FROM_DATE,
        "to-date": TO_DATE,
        "show-fields": "wordcount,sectionName,byline",
        "page-size": 200,
        "page": page,
        "api-key": API_KEYS[current_key]
    }
def _parse_slug(slug):
    if slug.startswith("section:"):
        return {"section": slug.replace("section:", "")}
    return {"tag": f"world/{slug}"}
def _parse_article(a, country_name):
    return {
        "country": country_name,
        "date": a["webPublicationDate"],
        "section": a["sectionName"],
        "wordcount": a.get("fields", {}).get("wordcount", None),
    }

```

```

        "title": a["webTitle"],
        "pillar": a.get("pillarName", None),
        "byline": a.get("fields", {}).get("byline", None),
    }
def fetch_articles(country_name, slug):
    global current_key
    articles, page = [], 1
    query_param = _parse_slug(slug)
    while True:
        r = requests.get(BASE_URL, params=_build_params(query_param, page))
        time.sleep(0.1)
        if r.status_code == 429:
            current_key = (current_key + 1) % len(API_KEYS)
            continue
        if r.status_code != 200:
            break
        data = r.json()["response"]
        results = data.get("results", [])
        if not results: break
        articles.extend([_parse_article(a, country_name) for a in results])
        if page >= data["pages"]: break
        page += 1
    return articles
def fetch_articles_slow(country_name, slug):
    global current_key
    articles, page = [], 1
    query_param = _parse_slug(slug)
    while True:
        try:
            r = requests.get(BASE_URL, params=_build_params(query_param, page))
            time.sleep(0.5)
        except requests.exceptions.Timeout:
            time.sleep(5)
            continue
        if r.status_code == 429:
            current_key = (current_key + 1) % len(API_KEYS)
            time.sleep(2)
            continue
        if r.status_code != 200: break
        data = r.json()["response"]
        results = data.get("results", [])
        if not results: break
        articles.extend([_parse_article(a, country_name) for a in results])
        if page % 50 == 0:
            print(f" ...page {page}, {len(articles)} articles so far")
        if page >= data["pages"]: break
        page += 1
    return articles

```

Step 3: We now run the main collection loop for the listed ACLED countries making sure that if the raw data file already exists, it loads from disk rather than re-querying the API. This creates the `guardian_raw.csv` file.

```

In [74]: RAW_PATH = "../data/guardian_raw.csv"
if os.path.exists(RAW_PATH) and os.path.getsize(RAW_PATH) > 0:
    print("Guardian raw data already collected – loading from file.")
    guardian_raw = pd.read_csv(RAW_PATH)
else:
    all_articles = []
    for country_name, slug in COUNTRY_LIST:
        print(f"Fetching: {country_name}...")

```

```

articles = fetch_articles(country_name, slug)
all_articles.extend(articles)
print(f" -> {len(articles)} articles")

guardian_raw = pd.DataFrame(all_articles)
guardian_raw.to_csv(RAW_PATH, index=False)
print(f"\nSaved {len(guardian_raw)} total articles to {RAW_PATH}")
print(f"\nShape: {guardian_raw.shape}")
print(f"Countries with articles: {guardian_raw['country'].nunique()}")
print(f>Date range: {guardian_raw['date'].min()} to {guardian_raw['date'].max()}")
guardian_raw.head()

```

Guardian raw data already collected – loading from file.

Shape: (276579, 7)

Countries with articles: 197

Date range: 2015-01-01T01:13:56Z to 2024-12-31T22:36:11Z

Out[74]:

	country	date	section	wordcount	title	pillar	by
0	Afghanistan	2024-12-31T18:41:27Z	UK news	495	Special forces troops face prosecution over al...	News	Kiran St. Pol correspon
1	Afghanistan	2024-12-29T17:09:11Z	Media	471	David Page obituary	News	Wi Cra
2	Afghanistan	2024-12-23T17:00:21Z	US news	2008	'Never underestimate what neighbors can do for...	News	David V
3	Afghanistan	2024-12-06T17:50:37Z	Global development	446	Taliban move to ban women training as nurses a...	News	Ruchi Ku and Z Joy Rukhsl M
4	Afghanistan	2024-12-06T10:40:29Z	Sport	471	Top Afghan cricketers urge Taliban to reverse ...	Sport	Jack Si

Step 4: Due to the volume of articles in their dedicated Guardian sections, the USA, UK, and Australia are collected separately using the slower function and added to the guardian_raw.csv.

```

In [ ]: aus_articles = fetch_articles_slow("Australia", "section:australia-news")
print(f"Australia: {len(aus_articles)} articles")

```

```

...page 50, 10000 articles so far
...page 100, 20000 articles so far
...page 150, 30000 articles so far
Error 400 on page 191 – stopping
Australia: 38000 articles

```

```

In [ ]: existing = pd.read_csv("../data/guardian_raw.csv")
aus_df = pd.DataFrame(aus_articles)
guardian_raw = pd.concat([existing, aus_df], ignore_index=True)
guardian_raw.to_csv("../data/guardian_raw.csv", index=False)
print(f"Total after Australia: {len(guardian_raw)} articles")

```

Total after Australia: 201282 articles

```
In [ ]: uk_articles = fetch_articles_slow("United Kingdom", "section:uk-news")
        print(f"UK: {len(uk_articles)} articles")

        ...page 50, 10000 articles so far
        ...page 100, 20000 articles so far
        ...page 150, 30000 articles so far
UK: 37297 articles
```

```
In [ ]: existing = pd.read_csv("../data/guardian_raw.csv")
        uk_df = pd.DataFrame(uk_articles)
        guardian_raw = pd.concat([existing, uk_df], ignore_index=True)
        guardian_raw.to_csv("../data/guardian_raw.csv", index=False)
        print(f"Total after UK: {len(guardian_raw)} articles")
```

Total after UK: 238579 articles

```
In [ ]: us_articles = fetch_articles_slow("United States", "section:us-news")
        print(f"US: {len(us_articles)} articles")

        ...page 50, 10000 articles so far
        ...page 100, 20000 articles so far
        ...page 150, 30000 articles so far
        Error 400 on page 191 – stopping
US: 38000 articles
```

```
In [ ]: existing = pd.read_csv("../data/guardian_raw.csv")
        us_df = pd.DataFrame(us_articles)
        guardian_raw = pd.concat([existing, us_df], ignore_index=True)
        guardian_raw.to_csv("../data/guardian_raw.csv", index=False)
        print(f"Total after USA: {len(guardian_raw)} articles")
```

Total after USA: 276579 articles

3. Preparation of the Data for Analysis

We have successfully loaded our three datasets. Before moving to the Exploratory Data Analysis, we first conduct an Initial Data Analysis (IDA) to assess data quality, completeness, and suitability. We do this separately for each dataset.

IDA for The Guardian, ACLED and World Bank income files

We first look at The Guardian dataset.

```
In [36]: print(f"Shape: {guardian_raw.shape}")
        print(f"\nDtypes:\n{guardian_raw.dtypes}")
        print(f"\nMissing values:\n{guardian_raw.isnull().sum()}")
        print(f"\nDuplicate rows: {guardian_raw.duplicated().sum()}")
        print(f"Duplicate titles: {guardian_raw.duplicated(subset='title').sum()}")
        print(f"\nSample duplicate titles:")
        print(guardian_raw[guardian_raw.duplicated(subset='title', keep=False)][['title']])
        print(f"\nSection distribution:\n{guardian_raw['section'].value_counts().head(5)}")
        print(f"\nDate sample:\n{guardian_raw['date'].sample(5)}")
        print(f"\nLowest coverage countries:\n{guardian_raw['country'].value_counts().head(5)}")
```



```
guardian_raw['year'] = pd.to_datetime(guardian_raw['date']).dt.year
print(f"\nArticles per year:\n{guardian_raw['year'].value_counts().sort_index}")
```

Shape: (276579, 9)

Dtypes:
country object
date datetime64[ns, UTC]
section object
wordcount int64
title object
pillar object
byline object
year int32
month int32
dtype: object

Missing values:

country 0
date 0
section 0
wordcount 0
title 0
pillar 1101
byline 2189
year 0
month 0
dtype: int64

Duplicate rows: 0

Duplicate titles: 50500

Sample duplicate titles:

title

What happened in the Russia-Ukraine war this week? Catch up with the must-read news and analysis 178

Coronavirus in the Pacific: weekly briefing 54

European elections 2019: Tories unlikely to deliver on Brexit, says Farage – as it happened 27

Name: count, dtype: int64

Section distribution:

section

World news 84890

US news 43716

Australia news 42085

UK news 41032

Opinion 13245

Name: count, dtype: int64

Date sample:

251752 2022-10-25 18:41:20+00:00

22448 2020-06-23 00:46:46+00:00

153962 2022-10-11 17:57:40+00:00

96522 2016-07-13 02:52:23+00:00

99051 2015-05-18 05:23:17+00:00

Name: date, dtype: datetime64[ns, UTC]

Lowest coverage countries:

country

San Marino 9

Liechtenstein 7

Andorra 5

Comoros 5

Sao Tome and Principe 5
Name: count, dtype: int64

Articles per year:

year	
2015	24888
2016	21280
2017	24875
2018	25527
2019	24802
2020	26265
2021	26355
2022	35129
2023	33595
2024	33863

Name: count, dtype: int64

The Guardian dataset contains 276,579 rows and 7 columns (country, date, section, wordcount, title, pillar, byline). As expected, wordcount is an integer while all other columns are strings.

Pillar and byline have 1,101 and 2,189 missing values respectively, but at 0.4% and 0.8% these are negligible. The 50,500 duplicate titles aren't real duplicates — The Guardian simply reuses generic titles like weekly Ukraine or Covid briefings. No rows need dropping.

World news dominates with 84,890 articles, consistent with our world/country tag approach. US news, Australia news, and UK news follow, reflecting the three countries queried by section. Dates are in consistent ISO format and will be converted to datetime in the cleaning step.

The lowest-coverage countries are small island states like Andorra and San Marino with under 10 articles each, reflecting their minimal conflict activity rather than any collection issue. Finally, there's a clear spike in 2022 (35,128 articles vs ~25,000 in prior years) that persists into 2023–24, almost certainly driven by Ukraine and Gaza.

```
In [31]: print(f"Shape: {acled.shape}")
print(f"\nDtypes:\n{acled.dtypes}")
print(f"\nMissing values:\n{acled.isnull().sum()}")
print(f"Duplicate country-year pairs: {acled.duplicated(subset=['COUNTRY', 'DATE']).sum()}")
print(f"\nCountries per year:\n{acled['YEAR'].value_counts().sort_index()}")
print(f"\nFatalities summary:\n{acled['FATALITIES'].describe()}")
print(f"Zero fatality rows: {(acled['FATALITIES'] == 0).sum()}")
print(f"\nRegion split:\n{acled['region_group'].value_counts()}")
```

Shape: (1634, 4)

Dtypes:
COUNTRY object
YEAR int64
FATALITIES int64
region_group object
dtype: object

Missing values:
COUNTRY 0
YEAR 0
FATALITIES 0
region_group 0
dtype: int64
Duplicate country-year pairs: 0

Countries per year:
YEAR
2015 61
2016 74
2017 77
2018 161
2019 161
2020 212
2021 245
2022 215
2023 213
2024 215
Name: count, dtype: int64

Fatalities summary:
count 1634.000000
mean 991.755202
std 4549.874372
min 0.000000
25% 0.000000
50% 4.000000
75% 150.000000
max 73573.000000
Name: FATALITIES, dtype: float64
Zero fatality rows: 566

Region split:
region_group
Global South 1221
Global North 413
Name: count, dtype: int64

The ACLED dataset contains 1,634 country-year observations across 4 columns, all correctly typed. No missing values or duplicate country-year pairs were found.

Coverage grows from 61 countries in 2015 to over 200 from 2020 onwards, reflecting ACLED's improving data collection rather than a genuine rise in conflict. This means the Global South may be slightly under-represented in earlier years which is a known limitation we acknowledge in the conclusion.

Fatalities range from 0 to 73,573 with a median of just 4 and a mean of 992, showing a handful of extremely severe conflicts pulling the average up. The maximum of 73,573 likely corresponds to Afghanistan or Syria at the peak of their conflicts. 566 rows record

zero fatalities showing conflict occurred but no deaths were reported. These are retained as zero is still a valid severity value.

```
In [37]: wb_latest = wb_latest.reset_index(drop=True)
print(f"Shape: {wb_latest.shape}")
print(f"\nSample:\n{wb_latest[['Entity', INCOME_COL]].head(7)}")
print(f"\nMissing in income column: {wb_latest[INCOME_COL].isnull().sum()}")
print(f"\nIncome distribution:\n{wb_latest[INCOME_COL].value_counts()}")
acled_names = set(acled['COUNTRY'].unique())
wb_names = set(wb_latest['Entity'].unique())
print(f"\nIn ACLED but not World Bank:\n{sorted(acled_names - wb_names)}")
```

Shape: (226, 4)

Sample:

	Entity	World Bank's income classification
0	Liberia	Low-income countries
1	Zambia	Lower-middle-income countries
2	Afghanistan	Low-income countries
3	Lesotho	Lower-middle-income countries
4	Libya	Upper-middle-income countries
5	Lebanon	Lower-middle-income countries
6	Liechtenstein	High-income countries

Missing in income column: 0

Income distribution:

World Bank's income classification	
High-income countries	90
Upper-middle-income countries	59
Lower-middle-income countries	51
Low-income countries	26

Name: count, dtype: int64

In ACLED but not World Bank:

['Akrotiri and Dhekelia', 'Anguilla', 'Antarctica', 'Bailiwick of Guernsey', 'Bailiwick of Jersey', 'British Indian Ocean Territory', 'Caribbean Netherlands', 'Christmas Island', 'Cocos (Keeling) Islands', 'Cook Islands', 'Falkland Islands', 'French Southern and Antarctic Lands', 'Guadeloupe', 'Heard Island and McDonald Islands', 'Ivory Coast', 'Martinique', 'Micronesia', 'Montserrat', 'Niue', 'Norfolk Island', 'Pitcairn', 'Reunion', 'Saint Helena, Ascension and Tristan da Cunha', 'Saint Pierre and Miquelon', 'Saint-Barthelemy', 'Saint-Martin', 'Sint Maarten', 'South Georgia and the South Sandwich Islands', 'Tokelau', 'United States Minor Outlying Islands', 'Vatican City', 'Virgin Islands, U.S.', 'Wallis and Futuna']

The World Bank dataset contains 226 countries across 4 columns with all four expected income categories present, confirming our binary Global North/South classification will work correctly.

No missing values were found. The 33 countries appearing in ACLED but not the World Bank are all small territories and dependencies and thus irrelevant to our analysis.

90 countries are classified as Global North (high income) and 136 as Global South across the remaining three income tiers, which aligns with the real-world distribution of nations.

Cleaning Process for The Guardian Dataset

The cleaning process for the cleaned_fatalities has already been done. Hence, we move on to The Guardian dataset.

As identified in the IDA, the date column is stored as a string and requires conversion before analysis. We also extract year and month from the date column to enable time series aggregation later.

```
In [75]: guardian_raw['date'] = pd.to_datetime(guardian_raw['date'], utc=True)
guardian_raw['year'] = guardian_raw['date'].dt.year
guardian_raw['month'] = guardian_raw['date'].dt.month
```

We now have two clean datasets: guardian_raw.csv and cleaned_fatalities.csv

Making the clean master datasets

Finally, to facilitate the later analysis we create two datasets. master.csv contains one row per country, aggregating total article count, average and median word count, total fatalities, section placement breakdown, and a normalised coverage intensity metric (articles per 1,000 fatalities). master_monthly.csv contains one row per country per month, enabling time series and responsiveness analysis. Both are saved to the /data folder.

```
In [76]: guardian_raw['wordcount'] = pd.to_numeric(guardian_raw['wordcount'], errors='coerce')
guardian_agg = guardian_raw.groupby('country').agg(
    total_articles = ('title', 'count'),
    avg_wordcount = ('wordcount', 'mean'),
    med_wordcount = ('wordcount', 'median'),
).reset_index()
acled_agg = acled.groupby('COUNTRY').agg(
    total_fatalities = ('FATALITIES', 'sum'),
    region_group = ('region_group', 'first')
).reset_index().rename(columns={'COUNTRY': 'country'})
master = guardian_agg.merge(acled_agg, on='country', how='inner')
section_breakdown = (
    guardian_raw
    .groupby(['country', 'section'])
    .size()
    .reset_index(name='count')
)
total_per_country = guardian_raw.groupby('country').size().reset_index(name='total')
section_breakdown = section_breakdown.merge(total_per_country, on='country')
section_breakdown['pct'] = section_breakdown['count'] / section_breakdown['total']
section_pivot = section_breakdown.pivot_table(
    index='country', columns='section', values='pct', fill_value=0
).reset_index()
master = master.merge(section_pivot, on='country', how='left')
master['articles_per_1000_fatalities'] = (
    master['total_articles'] / master['total_fatalities'] * 1000
).replace([float('inf'), -float('inf')], None)
master.to_csv("../data/master.csv", index=False)
guardian_raw['date'] = pd.to_datetime(guardian_raw['date'], utc=True)
guardian_raw['year'] = guardian_raw['date'].dt.year
guardian_raw['month'] = guardian_raw['date'].dt.month
guardian_monthly = guardian_raw.groupby(
    ['country', 'year', 'month'])
```

```

).agg(
    article_count = ('title', 'count'),
    avg_wordcount = ('wordcount', 'mean')
).reset_index()
acled_monthly = acled.copy()
acled_monthly['monthly_fatalities'] = acled_monthly['FATALITIES'] / 12
acled_monthly = acled_monthly.rename(columns={'COUNTRY': 'country', 'YEAR':
monthly = guardian_monthly.merge(
    acled_monthly[['country', 'year', 'monthly_fatalities', 'region_group']]
    on=['country', 'year'],
    how='left'
)
monthly.to_csv("../data/master_monthly.csv", index=False)

```

4. Exploratory Data Analysis

```

In [77]: country_map = (
    master[["country", "total_articles"]]
    .dropna(subset=["country", "total_articles"])
)
light_red_scale = [
    [0.0, "#feeceec"],
    [0.5, "#ee8484"],
    [1.0, "#eb6060"],
]
fig = px.choropleth(
    country_map,
    locations="country",
    locationmode="country names",
    color="total_articles",
    hover_name="country",
    color_continuous_scale=light_red_scale,
    title="Choropleth Map of Total Articles by Country",
)
fig.update_layout(
    paper_bgcolor="#e6e6e6",
    plot_bgcolor="#e6e6e6",
    geo=dict(
        showframe=False,
        showcoastlines=True,
        projection_type="natural earth",
        bgcolor="#e6e6e6",
    ),
)
fig.show()

```

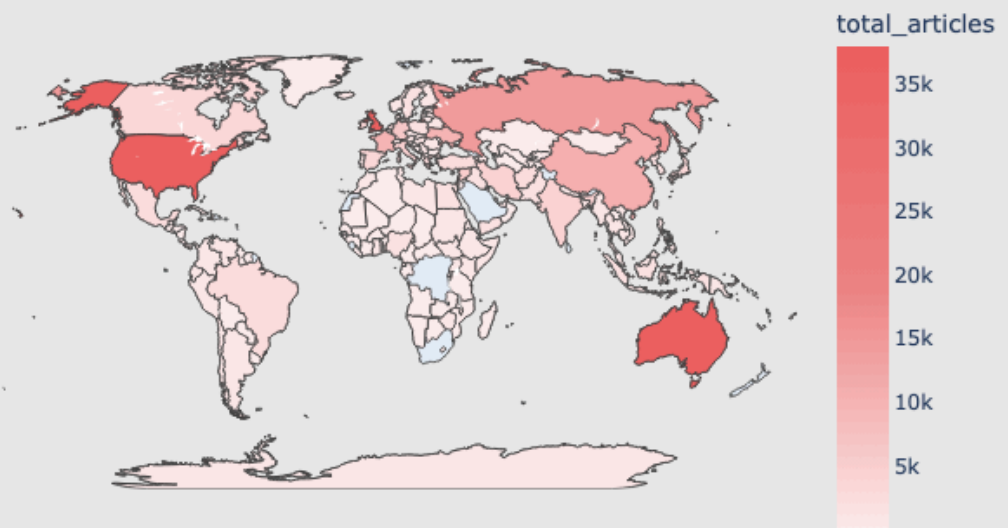
/var/folders/zz/6y2k3k_n28n203vcr45827br0000gp/T/ipykernel_4683/3919930600.py:10: DeprecationWarning:

The library used by the `*country names*` ``locationmode`` option is changing in an upcoming version. Country names in existing plots may not work in the new version. To ensure consistent behavior, consider setting ``locationmode`` to `*ISO-3*`.

Python(37891) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.

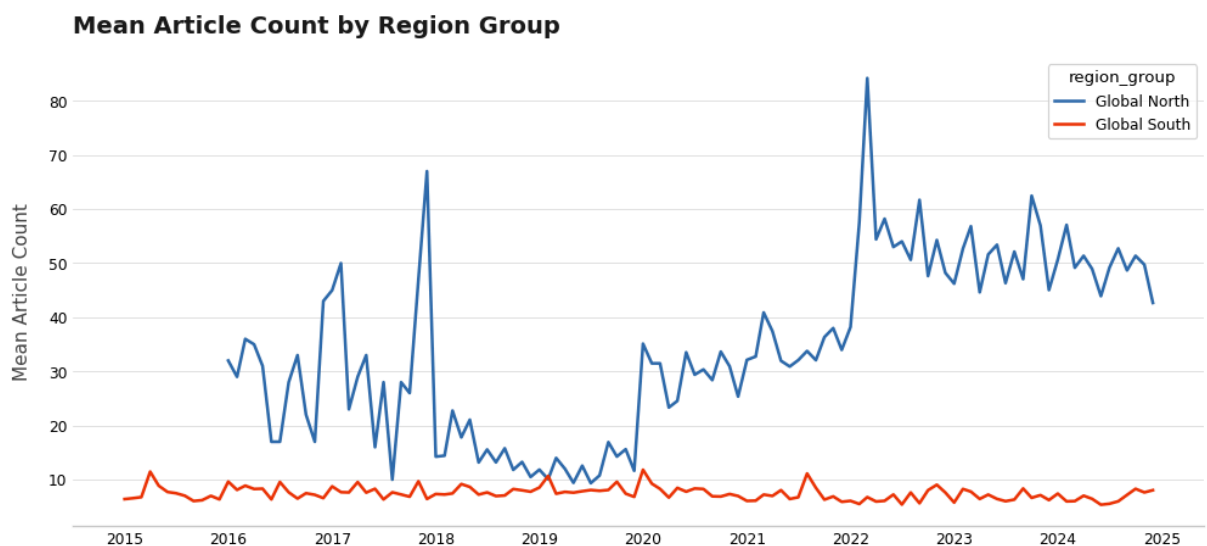
Resorting to unclean kill browser.

Choropleth Map of Total Articles by Country



We notice through this map Global North countries seem to consistently have a darker shade of red than most Global South countries. Let's dive deeper into this assumption.

```
In [78]: mean_fatalities_group = master_monthly.groupby(['region_group', 'date'])['a']
palette = ["#2D6AAB", "#EC3507"]
fig, ax = plt.subplots(figsize=(12, 5))
sns.lineplot(data=mean_fatalities_group, x="date", y="article_count", hue="region_group")
ax.grid(axis="y", color="#e5e5e5", linewidth=0.7, zorder=0)
ax.grid(axis="x", visible=False)
ax.spines[["top", "right", "left"]].set_visible(False)
ax.spines["bottom"].set_color("#cccccc")
ax.tick_params(axis="both", which="both", length=0)
ax.set_title("Mean Article Count by Region Group", fontsize=14, fontweight="bold")
ax.set_xlabel("", fontsize=0)
ax.set_ylabel("Mean Article Count", fontsize=11, color="#444444", labelpad=10)
plt.show()
```



Indeed, we see that Global North countries receive more article coverage than the Global South which in turn motivates our 3 research questions.

Research Question 1

Is there a measurable disparity in Guardian coverage volume between Global South and Global North countries, controlling for conflict severity?

Countries with zero fatalities in ACLED are excluded from this specific analysis, as the ratio is undefined.

Steps

1. Remove countries with 0 fatalities, log_transform total_articles and articles_per_1000_fatalities to ensure ANOVA normality assumptions

Scatter plot — log(fatalities) on the x-axis against log(article count) on the y-axis, colouring points by `region_group` (Global South vs. Global North).

2. Segment data according to fatality bins [0, 10, 100, 1000, 10000, float('inf')]
Box plot - of log_total_articles faceted by bin and coloured by region.

3. Two-way ANOVA with $FactorA = regiongroup$, $FactorB = severity\ bin$, and $DependentVariable = log(total_articles)$

ANOVA

Assumptions:

- Normality of residuals — use the Shapiro-Wilk test and a Q-Q plot on model residuals. With log-transformation this should be approximately satisfied.
- Homoscedasticity — use Levene's test across the region $\times$ bin groups. If this fails, switch to Welch's ANOVA (which doesn't assume equal variances) using `pingouin.welch_anova` or run the analysis with robust standard errors.
- Independence — countries are independent observations.

Reporting: F-statistic, degrees of freedom, p-value, and effect size (η^2). A significant p-value (<0.05) combined with a meaningful η^2 (>0.06 = medium) provides evidence of a systematic disparity.

Step 1: Data Preparation & Scatter plot: article count vs. fatalities by region

```
In [82]: match_df = master.copy()
match_df['log_articles'] = np.log1p(match_df['total_articles'])
match_df['log_fatalities'] = np.log1p(match_df['total_fatalities'])
match_df['severity_bin'] = pd.cut(
    match_df['total_fatalities'],
    bins=[0, 100, 1000, 10000, 50000, float('inf')],
    labels=['0-100', '100-1k', '1k-10k', '10k-50k', '50k+']
)
```

```
bin_labels = ['0-100', '100-1k', '1k-10k', '10k-50k', '50k+']
match_df = match_df.dropna(subset=['severity_bin'])
```

```
In [84]: scatter_df = master[master["total_fatalities"] > 0].copy()
scatter_df['log_fatalities'] = np.log1p(scatter_df['total_fatalities'])
scatter_df['log_articles'] = np.log1p(scatter_df['total_articles'])
color_map = {"Global South": "#e05c5c", "Global North": "#4c7bb5"}

fig = px.scatter(
    scatter_df,
    x="log_fatalities",
    y="log_articles",
    color="region_group",
    color_discrete_map=color_map,
    hover_name="country",
    hover_data={
        "total_articles": True,
        "total_fatalities": True,
        "log_fatalities": False,
        "log_articles": False,
        "region_group": False,
    },
    title="Guardian Article Count vs. Conflict Fatalities by Region (2015-2020)",
    labels={
        "log_fatalities": "log10(Total Fatalities)",
        "log_articles": "log10(Total Articles)",
        "region_group": "Region",
    },
)

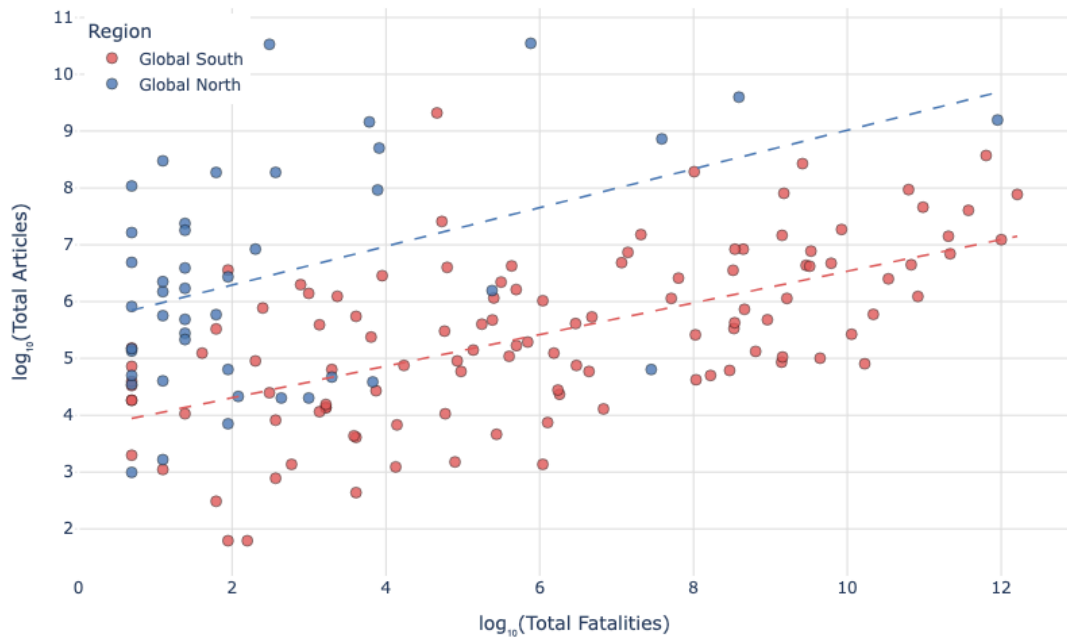
for group, color in color_map.items():
    grp = scatter_df[scatter_df["region_group"] == group]
    m, b = np.polyfit(grp["log_fatalities"], grp["log_articles"], 1)
    x_range = np.linspace(grp["log_fatalities"].min(), grp["log_fatalities"].max(), 10)
    fig.add_scatter(
        x=x_range,
        y=m * x_range + b,
        mode="lines",
        line=dict(color=color, dash="dash", width=1.5),
        showlegend=False,
        hoverinfo="skip",
    )

fig.update_traces(marker=dict(size=8, opacity=0.8, line=dict(width=0.5, color="black")),
                  selector=dict(mode="markers"))
fig.update_layout(
    width=900,
    height=600,
    legend=dict(title="Region", x=0.01, y=0.99),
    xaxis=dict(showgrid=True, gridcolor="#e0e0e0"),
    yaxis=dict(showgrid=True, gridcolor="#e0e0e0"),
    plot_bgcolor="white",
)
fig.show()

print(scatter_df.groupby("region_group")[["total_articles", "total_fatalities",
                                           "articles_per_1000_fatalities"]].r
```

Python(38075) MallocStackLogging: can't turn off malloc stack logging because it was not enabled.
Resorting to unclean kill browser.

Guardian Article Count vs. Conflict Fatalities by Region (2015–2024)



	total_articles	total_fatalities	articles_per_1000_fatalities
Global North	489.0	5.0	106145.251397
Global South	225.0	296.5	597.785021

Comment:

1. We can see that the despite lower fatalities, Global North countries get more coverage overall, as shown by the fitted line.
2. Global south countries lie at the lower half the the region

Step 2. Matched comparison: countries with similar fatalities, different regions

```
In [85]: %matplotlib inline

def styled_box(ax, data, positions, color, medcolor, label):
    bp = ax.boxplot(
        data,
        positions=positions,
        widths=width,
        patch_artist=True,
        notch=False,
        showfliers=True,
        boxprops=dict(facecolor=color, alpha=0.75, linewidth=1.2),
        medianprops=dict(color=medcolor, linewidth=2.2),
        whiskerprops=dict(color=color, linewidth=1.2, linestyle='--'),
        capprops=dict(color=color, linewidth=1.5),
        flierprops=dict(marker='o', markerfacecolor=color, markeredgecolor=
                        markersize=5, alpha=0.7, linewidth=0.5),
        manage_ticks=False,
        label=label
    )
    return bp

cols = {'Global South': '#1D9E75', 'Global North': '#7F77DD'}
medline = {'Global South': '#085041', 'Global North': '#3C3489'}
```

```

fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor('#F9F9F7')
ax.set_facecolor('#F9F9F7')

n_bins = len(bin_labels)
width = 0.32
gap = 0.06
positions_gs = np.arange(n_bins)
positions_w = np.arange(n_bins) + width + gap

all_data_gs, all_data_w = [], []
for bl in bin_labels:
    gs = match_df[(match_df['severity_bin']==bl) & (match_df['region_group']=='Global South')]
    w = match_df[(match_df['severity_bin']==bl) & (match_df['region_group']=='Global North')]
    all_data_gs.append(gs)
    all_data_w.append(w)

styled_box(ax, all_data_gs, positions_gs, cols['Global South'], medline['Global South'])
styled_box(ax, all_data_w, positions_w, cols['Global North'], medline['Global North'])

for i, (gs, w) in enumerate(zip(all_data_gs, all_data_w)):
    ax.text(positions_gs[i], 1.3, f'n={len(gs)}', ha='center', va='bottom',
            fontsize=7.5, color=medline['Global South'], fontweight='bold')
    ax.text(positions_w[i], 1.3, f'n={len(w)}', ha='center', va='bottom',
            fontsize=7.5, color=medline['Global North'], fontweight='bold')

tick_positions = (positions_gs + positions_w) / 2
ax.set_xticks(tick_positions)
ax.set_xticklabels(bin_labels, fontsize=10)

ax.set_ylabel('log(1 + total articles)', fontsize=11)
ax.set_xlabel('Conflict severity bin (total fatalities)', fontsize=11)
ax.set_title('Guardian coverage volume by conflict severity and region group',
            fontsize=13, fontweight='bold', pad=14)

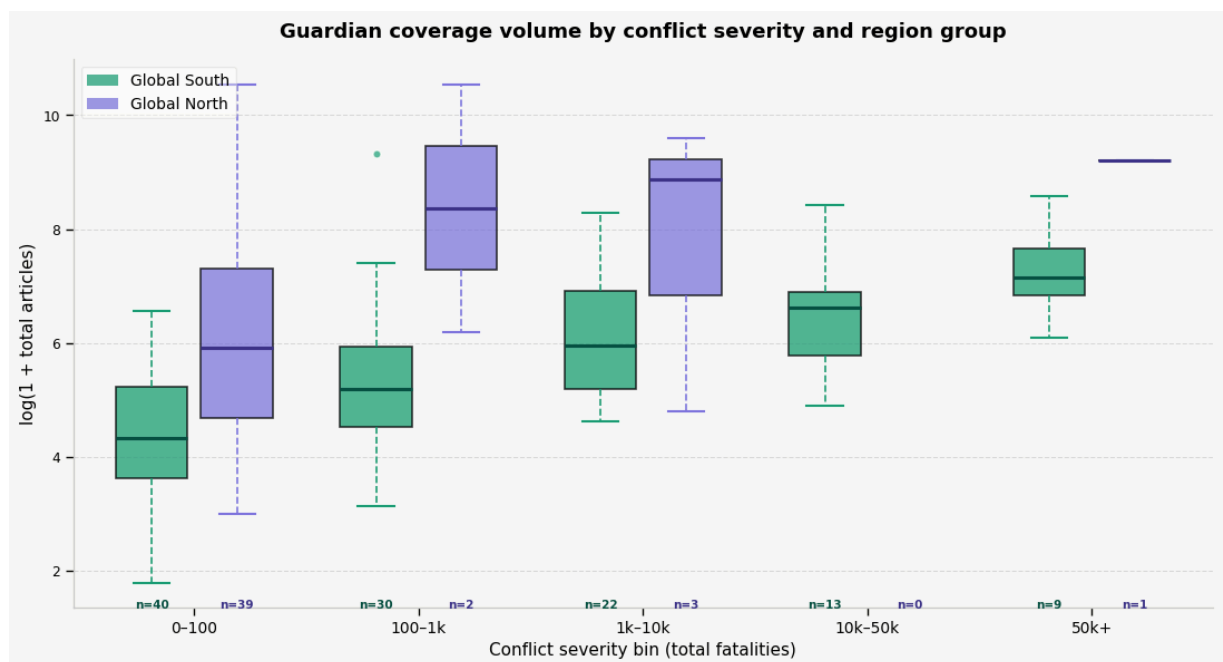
ax.yaxis.grid(True, linestyle='--', alpha=0.5, color='#BBBBBB')
ax.set_axisbelow(True)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
for spine in ['left', 'bottom']:
    ax.spines[spine].set_color('#CCCCCC')

patch_gs = mpatches.Patch(facecolor=cols['Global South'], alpha=0.75, label=medline['Global South'])
patch_w = mpatches.Patch(facecolor=cols['Global North'], alpha=0.75, label=medline['Global North'])
ax.legend(handles=[patch_gs, patch_w], fontsize=10, framealpha=0.4,
        loc='upper left', edgecolor='#CCCCCC')

ax.set_xlim(-0.35, n_bins - 1 + width + gap + 0.35)

plt.tight_layout()
plt.show()

```



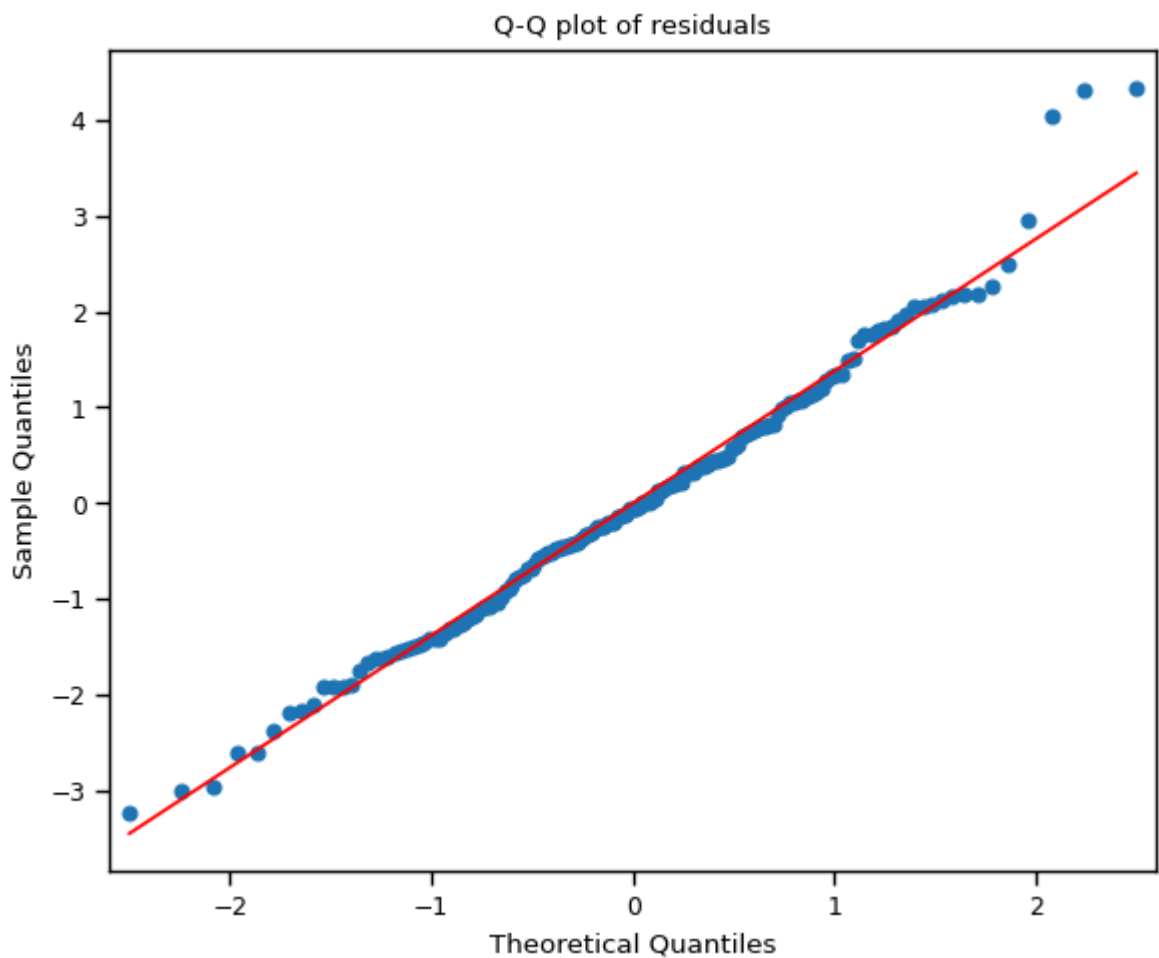
Comment:

- We have confirmed from box plot that for all fatality bins, the Global North box sits very high.
- However, the Global North country data points are too small (eg.9,2,3,1), which do not form meaningful boxes
- Given this

Step 3: ANOVA

- Check assumptions
 - Normality of residuals
 - Equal Variance

```
In [86]: model = smf.ols('log_articles ~ C(region_group) * C(severity_bin)', data=ma
residuals = model.resid
fig, ax = plt.subplots(figsize=(6, 5))
sm.qqplot(residuals, line='s', ax=ax)
ax.set_title('Q-Q plot of residuals')
plt.tight_layout()
plt.show()
```



```
In [87]: col = {'Global South': '#1D9E75', 'Global North': '#7F77DD'}

fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor('#F9F9F7')
ax.set_facecolor('#F9F9F7')
x = np.arange(len(bin_labels))

for region in ['Global South', 'Global North']:
    means, cis = [], []
    for bl in bin_labels:
        vals = match_df[
            (match_df['severity_bin'] == bl) &
            (match_df['region_group'] == region)
        ]['log_articles'].values

        n = len(vals)
        mean = vals.mean()
        se = stats.sem(vals)
        ci = se * stats.t.ppf(0.975, df=n-1)

        means.append(mean)
        cis.append(ci)

    means = np.array(means)
    cis = np.array(cis)

    ax.plot(x, means, marker='o', linewidth=2.2, markersize=7,
            color=col[region], label=region, zorder=3)

    ax.fill_between(x, means - cis, means + cis,
                    alpha=0.15, color=col[region], zorder=2)
```

```

ax.errorbar(x, means, yerr=cis, fmt='none',
            ecol=col[region], elinewidth=1.3, capsize=4, zorder=4)

ax.set_xticks(x)
ax.set_xticklabels(bin_labels, fontsize=10)
ax.set_ylabel('Mean log(1 + total articles)', fontsize=11)
ax.set_xlabel('Conflict severity bin (total fatalities)', fontsize=11)
ax.set_title(
    'Interaction plot: Guardian coverage by region and conflict severity\n(
    fontsize=12, fontweight='bold', pad=14
)

ax.yaxis.grid(True, linestyle='--', alpha=0.45, color='#BBBBBB')
ax.set_axisbelow(True)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
for spine in ['left', 'bottom']:
    ax.spines[spine].set_color('#CCCCCC')

ax.legend(fontsize=10, framealpha=0.4, edgecolor='#CCCCCC', loc='upper left')

for i, bl in enumerate(bin_labels):
    gs_mean = match_df[
        (match_df['severity_bin'] == bl) &
        (match_df['region_group'] == 'Global South')
    ]['log_articles'].mean()

    w_mean = match_df[
        (match_df['severity_bin'] == bl) &
        (match_df['region_group'] == 'Global North')
    ]['log_articles'].mean()

    gap = w_mean - gs_mean
    ax.annotate(
        f' $\Delta$ ={gap:.2f}',
        xy=(i, (gs_mean + w_mean) / 2),
        ha='center', va='center', fontsize=8, color='#888888',
        bbox=dict(boxstyle='round,pad=0.2', facecolor='white',
                  edgecolor='#CCCCCC', alpha=0.7)
    )

plt.tight_layout()

```

```
/var/folders/zz/6y2k3k_n28n203vcr45827br0000gp/T/ipykernel_4683/3181890592.py:17: RuntimeWarning:
```

Mean of empty slice.

```
/Users/celine/Library/Python/3.9/lib/python/site-packages/numpy/_core/_methods.py:138: RuntimeWarning:
```

invalid value encountered in scalar divide

```
/Users/celine/Library/Python/3.9/lib/python/site-packages/numpy/_core/_methods.py:218: RuntimeWarning:
```

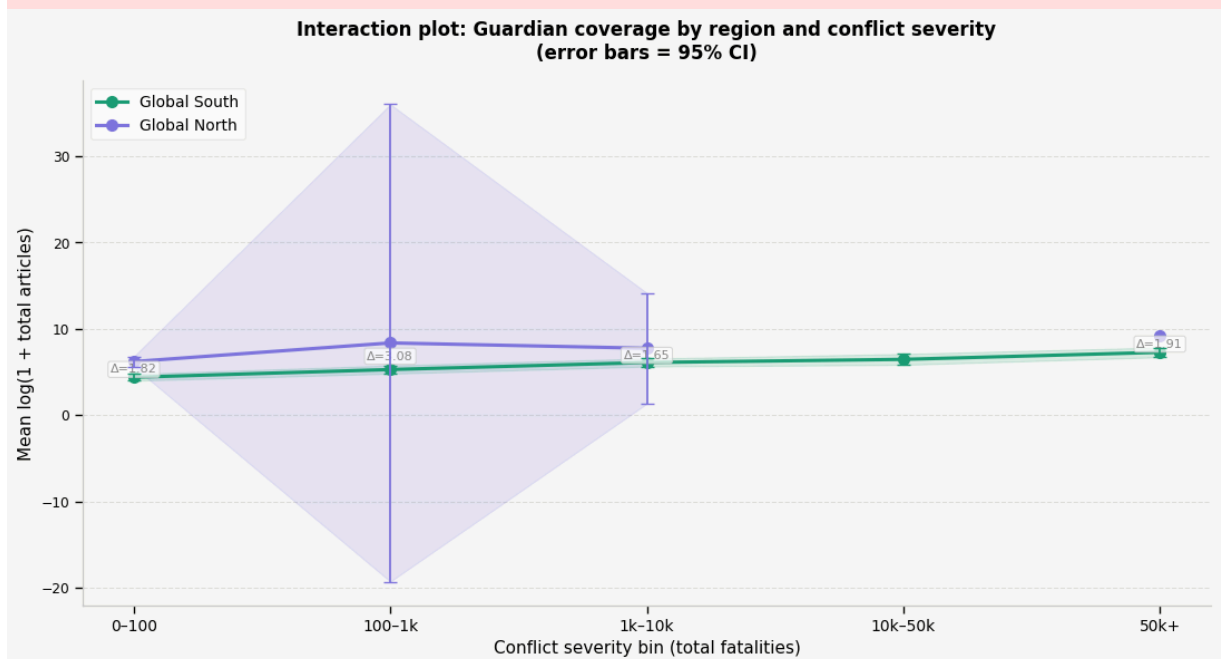
Degrees of freedom ≤ 0 for slice

```
/Users/celine/Library/Python/3.9/lib/python/site-packages/numpy/_core/_methods.py:175: RuntimeWarning:
```

invalid value encountered in divide

```
/Users/celine/Library/Python/3.9/lib/python/site-packages/numpy/_core/_methods.py:210: RuntimeWarning:
```

invalid value encountered in scalar divide



1. There is a strong, statistically significant coverage disparity between Global North and Global South countries that cannot be explained by conflict severity (region $F=68.25$, $p \approx 0$).
2. Conflict severity itself showed significant results (severity $F=32.96$, $p \approx 0$), confirming the control for severity is meaningful.
3. We fail to reject the null hypothesis that the regional coverage gap is the same across all severity bins.
 - This means it is consistent across all severity bins that coverage on Global North countries is higher.

Research Question 2

Do structural features of articles — length and section placement — differ systematically by region?

For **article length**, the question is whether Global South articles are systematically shorter than Global North articles. For **section placement**, the question is whether Global South coverage concentrates in brief world-news mentions while Global North stories more often appear in Opinion or Politics.

Steps

1. **Boxplots** — Plot the distribution of `med_wordcount` by `region_group`. Median word count is more informative than mean because word count is right-skewed.
2. Use Mann-Whitney U to test whether one group systematically have higher median wordcounts.

H_0 : The distribution of `med_wordcount` is the same for Global South and Global North countries. A randomly chosen Global South country is equally likely to have a higher or lower word count than a randomly chosen Global North country.

```
In [88]: col      = {'Global South': '#1D9E75', 'Global North': '#7F77DD'}
medline = {'Global South': '#085041', 'Global North': '#3C3489'}
regions = ['Global South', 'Global North']

box_df = [master[region_group == r]['med_wordcount'].dropna().value_counts()
          for region_group in regions]

fig, ax = plt.subplots(figsize=(7, 6))
fig.patch.set_facecolor('#F9F9F7')
ax.set_facecolor('#F9F9F7')

bp = ax.boxplot(
    box_df,
    positions=[1, 2],
    widths=0.45,
    patch_artist=True,
    notch=False,
    showfliers=True,
    boxprops=dict(linewidth=1.2),
    medianprops=dict(linewidth=2.2),
    whiskerprops=dict(linewidth=1.2, linestyle='--'),
    capprops=dict(linewidth=1.5),
    flierprops=dict(marker='o', markeredgecolor='white', markersize=5, alpha=0.75),
    manage_ticks=False
)

for patch, region in zip(bp['boxes'], regions):
    patch.set_facecolor(col[region])
    patch.set_alpha(0.75)

for median, region in zip(bp['medians'], regions):
    median.set_color(medline[region])

for flier, region in zip(bp['fliers'], regions):
    flier.set_markerfacecolor(col[region])
```

```

for i, region in enumerate(regions):
    bp['whiskers'][i*2].set_color(col[region])
    bp['whiskers'][i*2 + 1].set_color(col[region])
    bp['caps'][i*2].set_color(col[region])
    bp['caps'][i*2 + 1].set_color(col[region])

np.random.seed(42)
for pos, vals, region in zip([1, 2], box_df, regions):
    jitter = np.random.uniform(-0.12, 0.12, size=len(vals))
    ax.scatter(pos + jitter, vals, alpha=0.35, s=18,
               color=col[region], edgecolors='white', linewidths=0.4, zorder=2)

for pos, vals, region in zip([1, 2], box_df, regions):
    ax.text(pos, master['med_wordcount'].min() - 18, f'n={len(vals)}',
            ha='center', fontsize=9, color=medline[region], fontweight='bold')
    ax.text(pos + 0.26, np.median(vals) + 2, f'med={np.median(vals):.0f}',
            ha='left', fontsize=8, color=medline[region])

ax.set_xticks([1, 2])
ax.set_xticklabels(regions, fontsize=11)
ax.set_ylabel('Median word count per article', fontsize=11)
ax.set_title('Distribution of article word count\nby region group', fontsize=11)
ax.set_xlim(0.5, 2.7)
ax.set_ylim(master['med_wordcount'].min() - 40, master['med_wordcount'].max() + 40)

ax.yaxis.grid(True, linestyle='--', alpha=0.45, color='#BBBBBB')
ax.set_axisbelow(True)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
for spine in ['left', 'bottom']:
    ax.spines[spine].set_color('#CCCCCC')

plt.tight_layout()

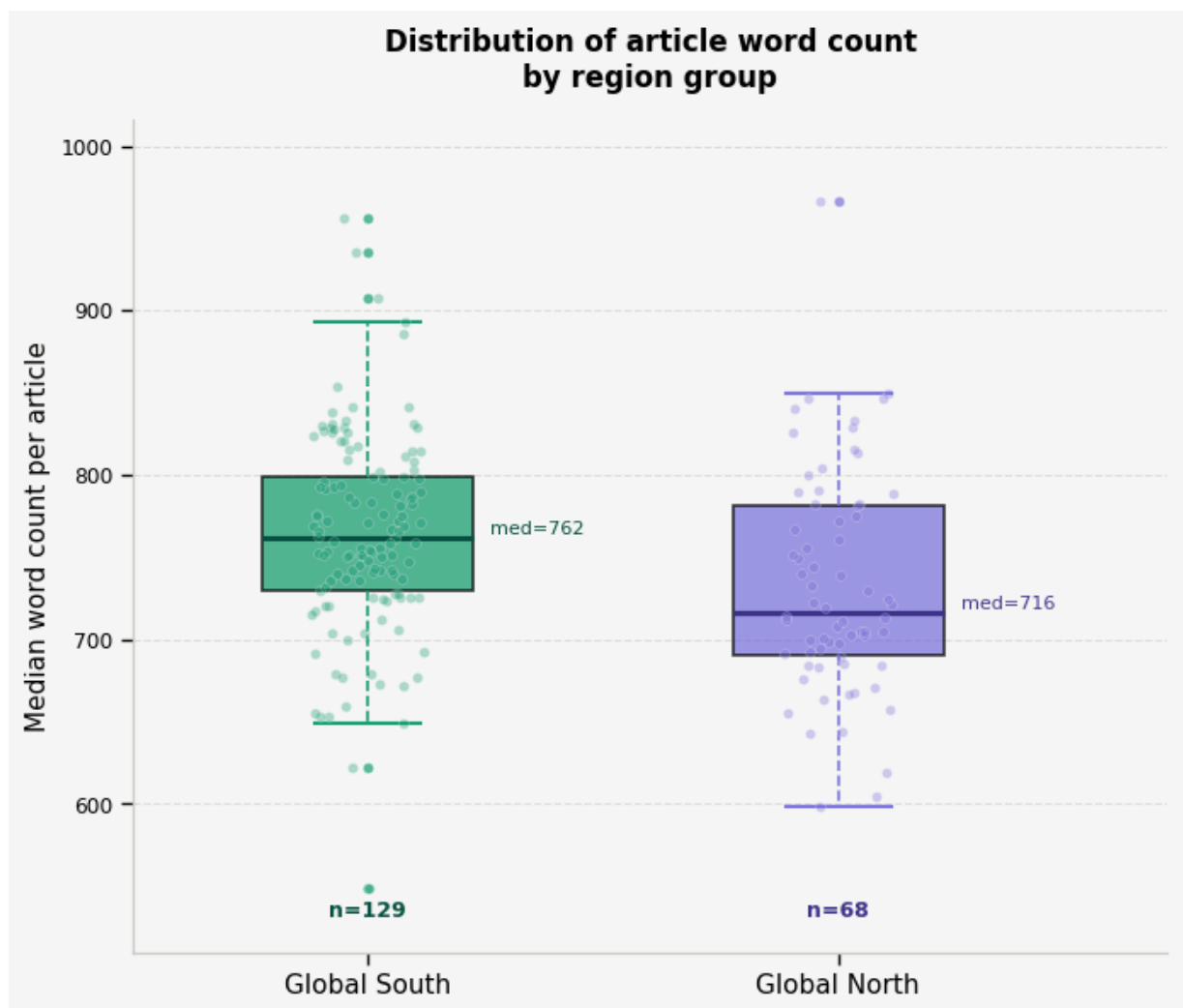
gs = master[master['region_group'] == 'Global South']['med_wordcount'].dropna()
w = master[master['region_group'] == 'Global North']['med_wordcount'].dropna()
u_scipy, p_scipy = stats.mannwhitneyu(gs, w, alternative='two-sided')
print(f'p-value (scipy): {p_scipy:.6f}')
print(f'U (scipy): {u_scipy}')

```

```

p-value (scipy): 0.000282
U (scipy): 5768.0

```



We reject the null hypothesis given that the p-value is very small

- Specifically, Global South articles have a higher median word count (761 vs 717 words)

Research Question 3:

Does The Guardian respond to casualty spikes differently depending on region, and has any disparity narrowed or widened over time (2015–2024)?

To answer this question, we first check how many countries appear in both the top 10 for casualties and the top 10 for Guardian articles. We then calculate the Pearson correlation coefficient to measure the strength of the linear relationship between these two variables in each region, and track how this correlation changes over time from 2015 to 2024. We use Pearson correlation because we are testing a linear relationship between two continuous, quantitative measures. It also allows us to compare responsiveness across regions, as the coefficient directly indicates the strength of the relationship.

Step 1: Prepare the data for analysis.

```
In [89]: monthly = pd.read_csv('../data/master_monthly.csv')
master   = pd.read_csv('../data/master.csv')
wb       = pd.read_csv('../data/worldbank_income_clean.csv')
```

```

wb_region = wb[['country', 'region_group']].drop_duplicates()
monthly = monthly.drop(columns='region_group')
monthly = monthly.merge(wb_region, on='country', how='inner')
master = master.drop(columns='region_group')
master = master.merge(wb_region, on='country', how='inner')
df = monthly[monthly['monthly_fatalities'].notna()].copy()
yearly_agg = df.groupby(['year', 'region_group']).agg(
    total_articles = ('article_count', 'sum'),
    total_fatalities = ('monthly_fatalities', 'sum')
).reset_index()

```

Step 2: Prepare graph plotting function for analysis

```

In [90]: def build_yearly_scatter(yearly, title):
        """
        Side-by-side scatter: Global South | Global North.
        One point per year, with Pearson r and OLS trend line.
        Prints stats when called.
        """
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

        print(f"\n{'='*60}")
        print(f"PEARSON - {title}")
        print(f"{'='*60}")

        for ax, (region, color) in zip([ax1, ax2], [
            ('Global South', '#E07A5F'),
            ('Global North', '#3D405B')
        ]):
            sub = yearly[yearly['region_group'] == region]
            if len(sub) < 3:
                continue

            r_val, p_val = pearsonr(sub['total_fatalities'], sub['total_articles'])
            print(f"    {region}:    r = {r_val:.4f}, p = {p_val:.4e}, n = {len(sub)}")

            ax.scatter(sub['total_fatalities'], sub['total_articles'],
                        color=color, s=80, zorder=3)
            for _, row in sub.iterrows():
                ax.annotate(str(int(row['year'])),
                            (row['total_fatalities'], row['total_articles']),
                            textcoords='offset points', xytext=(0, 8),
                            fontsize=8, ha='center')

            z = np.polyfit(sub['total_fatalities'], sub['total_articles'], 1)
            x_line = np.linspace(sub['total_fatalities'].min(),
                                sub['total_fatalities'].max(), 100)
            ax.plot(x_line, np.polyval(z, x_line), '--', color=color, linewidth=2)

            ax.text(0.95, 0.95, f"r = {r_val:.3f}, p = {p_val:.2e}",
                    transform=ax.transAxes, fontsize=11, ha='right', va='top',
                    bbox=dict(boxstyle='round,pad=0.4', facecolor='white',
                              edgecolor=color, alpha=0.8))

            ax.set_title(region, fontsize=12, fontweight='bold')
            ax.set_xlabel('Yearly Casualties', fontsize=10)
            ax.grid(True, alpha=0.3)
            y_min, y_max = ax.get_ylim()
            y_pad = (y_max - y_min) * 0.08
            ax.set_ylim(y_min - y_pad, y_max + y_pad)

        ax1.set_ylabel('Yearly Articles', fontsize=10)

```

```

fig.suptitle(title, fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

def build_scatter(master_data, region, color, title):
    """
    Scatter plot of casualties vs articles for one region (country-level).
    Prints Pearson r when called.
    """
    sub = master_data[
        (master_data['region_group'] == region) &
        (master_data['total_fatalities'].notna()) &
        (master_data['total_fatalities'] > 0)
    ].copy()

    r_val, p_val = pearsonr(sub['total_fatalities'], sub['total_articles'])
    print(f"\n{region} country-level Pearson: r = {r_val:.4f}, p = {p_val:.4f}")

    fig, ax = plt.subplots(figsize=(10, 6))

    ax.scatter(sub['total_fatalities'], sub['total_articles'],
               color=color, s=50, alpha=0.7, zorder=3)

    for _, row in sub.iterrows():
        ax.annotate(row['country'],
                    (row['total_fatalities'], row['total_articles']),
                    textcoords='offset points', xytext=(0, 6),
                    fontsize=7, ha='center', color='grey')

    # OLS trend line
    z = np.polyfit(sub['total_fatalities'], sub['total_articles'], 1)
    x_line = np.linspace(sub['total_fatalities'].min(), sub['total_fatalities'].max(), 100)
    ax.plot(x_line, np.polyval(z, x_line), '--', color=color, linewidth=2)

    ax.text(0.95, 0.95, f"r = {r_val:.3f}, p = {p_val:.2e}, n = {len(sub)}",
            transform=ax.transAxes, fontsize=12, ha='right', va='top',
            bbox=dict(boxstyle='round,pad=0.4', facecolor='white',
                      edgecolor=color, alpha=0.8))

    ax.set_title(title, fontsize=13, fontweight='bold')
    ax.set_xlabel('Total Casualties', fontsize=11)
    ax.set_ylabel('Total Articles', fontsize=11)
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

def build_pearson_by_year(monthly_data):
    """
    Line chart of Pearson r by year for both regions.
    Prints correlation stats when called.
    """
    df = monthly_data[monthly_data['monthly_fatalities'].notna()].copy()

    yearly = df.groupby(['country', 'year', 'region_group']).agg(
        total_articles = ('article_count', 'sum'),
        total_fatalities = ('monthly_fatalities', 'sum')
    ).reset_index()

    results = []
    for region in ['Global South', 'Global North']:

```

```

for year in sorted(yearly['year'].unique()):
    sub = yearly[(yearly['region_group'] == region) & (yearly['year']
    if len(sub) > 2:
        r_val, p_val = pearsonr(sub['total_fatalities'], sub['total_
        countries = sorted(sub['country'].tolist())
        results.append({
            'year': year,
            'region_group': region,
            'r': r_val,
            'p': p_val,
            'n': len(sub),
            'significant': p_val < 0.05,
            'countries': countries
        })

results_df = pd.DataFrame(results)

print(f"\n{'='*60}")
print("PEARSON r BY YEAR")
print(f"{'='*60}")
for _, row in results_df.iterrows():
    sig = '***' if row['p'] < 0.001 else '**' if row['p'] < 0.01 else '
    print(f"  {row['region_group']:15s} {row['year']}  r={row['r']:.3f}

gs = results_df[results_df['region_group'] == 'Global South']
gn = results_df[results_df['region_group'] == 'Global North']

fig, ax = plt.subplots(figsize=(10, 5.5))

for subset, name, color, y_offset in [
    (gs, 'Global South', '#E07A5F', 12),
    (gn, 'Global North', '#3D405B', -16)
]:
    sig_mask = subset['significant']

    ax.plot(subset['year'], subset['r'], '-', color=color, linewidth=2,
    if sig_mask.any():
        ax.scatter(subset.loc[sig_mask, 'year'], subset.loc[sig_mask, '
            color=color, s=80, zorder=4, marker='o')
    if (~sig_mask).any():
        ax.scatter(subset.loc[~sig_mask, 'year'], subset.loc[~sig_mask,
            facecolors='white', edgecolors=color, s=80, zorder=4
            marker='o', linewidths=1.5)
    for _, row in subset.iterrows():
        ax.annotate(f"n={row['n']}", (row['year'], row['r']),
            textcoords='offset points', xytext=(0, y_offset),
            fontsize=8, ha='center', color=color)
ax.axhline(y=0, linestyle=':', color='grey', alpha=0.5)
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Pearson r', fontsize=11)
ax.set_title('Pearson Correlation (Casualties vs Articles) by Year and I
            fontsize=13, fontweight='bold')
ax.set_ylim(-0.3, 1.0)
ax.set_xticks(range(2015, 2025))
ax.legend(loc='lower left', framealpha=0.8)
ax.grid(True, alpha=0.3)
fig.text(0.5, -0.02,
        '● filled = significant (p < 0.05)   ○ open = not significant'
        ha='center', fontsize=10, color='grey')
plt.tight_layout()
plt.show()

```

Step 3: We will now plot graph for our analysis

Graph 1

```
In [91]: build_yearly_scatter(  
    yearly_agg,  
    'Yearly: The Guardian articles vs ACLED casualties by region'  
)
```

=====

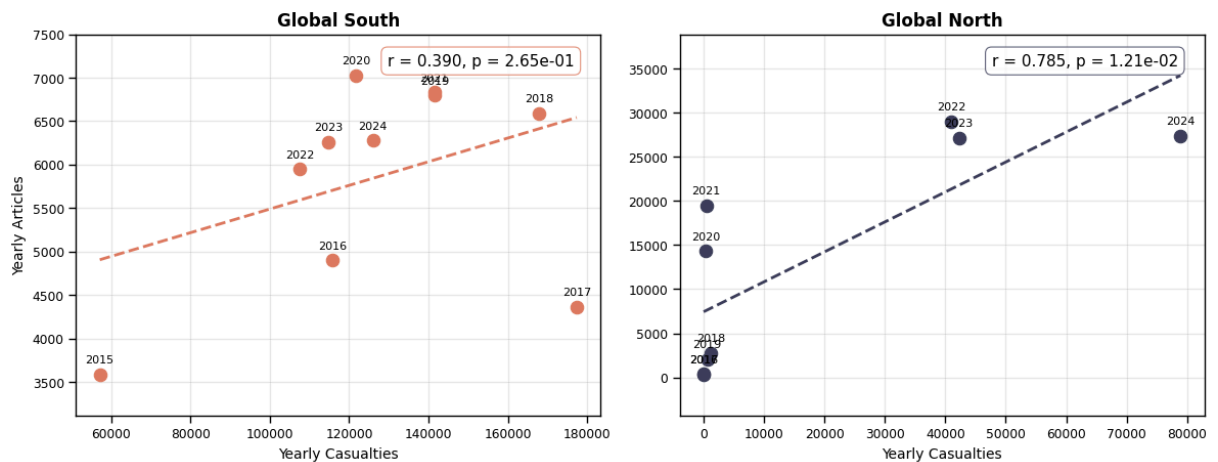
PEARSON – Yearly: The Guardian articles vs ACLED casualties by region

=====

Global South: $r = 0.3905$, $p = 2.6457e-01$, $n = 10$

Global North: $r = 0.7854$, $p = 1.2135e-02$, $n = 9$

Yearly: The Guardian articles vs ACLED casualties by region

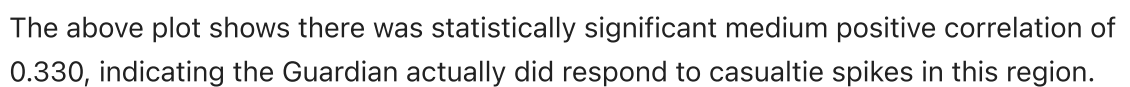


The scatter plot shows that these two variables have a stronger correlation in the Global North than in the Global South, with Pearson correlation coefficients of 0.786 and 0.391 respectively. Considering the p-values — 0.26 for the Global South and 0.012 for the Global North — we can conclude that the positive correlation between casualties and article count is statistically significant only for the Global North over the 2015–2024 period.

Graph 2

```
In [92]: build_scatter(  
    master, 'Global South', '#E07A5F',  
    'Global South: Total Casualties vs Total Articles by Country (2015–2024'  
)
```

Global South country-level Pearson: $r = 0.3312$, $p = 3.3952e-04$, $n = 113$



```
In [93]: build_scatter(
    master, 'Global North', '#3D405B',
    'Global North: Total Casualties vs Total Articles by Country (2015–2024)'
)
```

Surprisingly, this graph shows a different result compared to our first analysis. The scatter plot reveals that the correlation between casualties and articles in the Global North is statistically insignificant and very weak. This suggests that the correlation found in the first graph was coincidental, and that Guardian coverage of Global North countries may be driven by other factors — such as geopolitical importance or the

demographic profile of the Guardian's readership — rather than in direct response to casualty spikes.

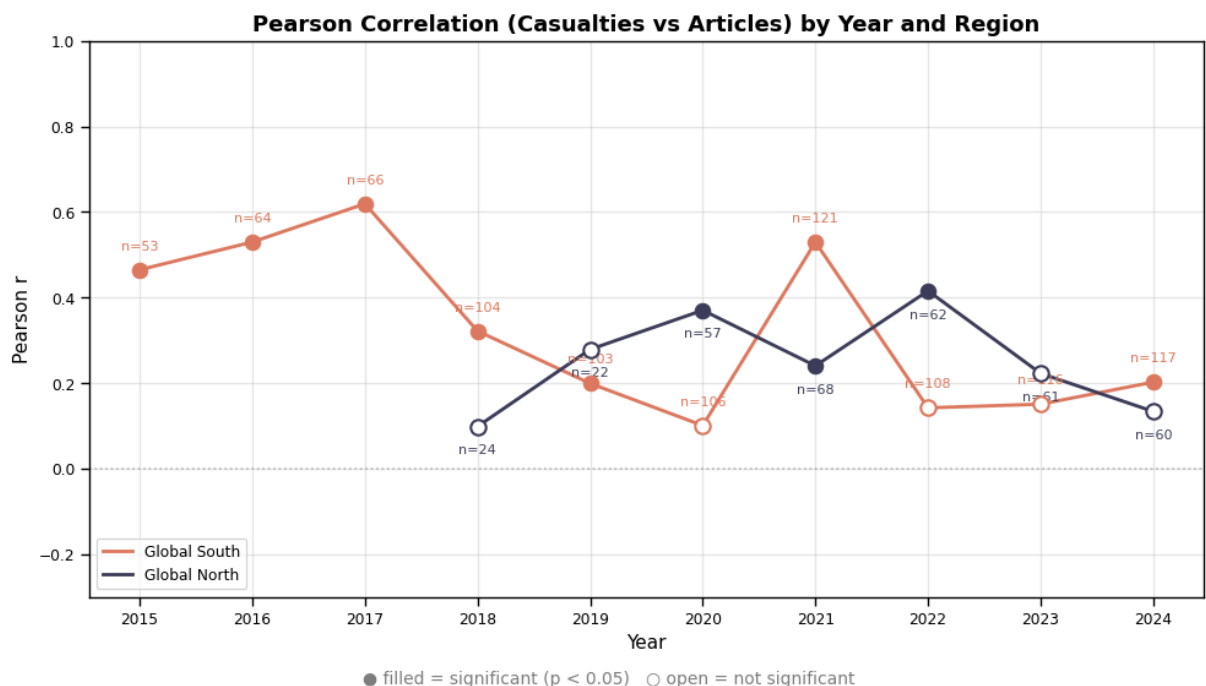
To examine how the Guardian's responsiveness to casualties has changed over time, we plot a line graph showing how the Pearson correlation evolves by year. Note that in some years, certain countries have no casualty data due to ACLED coverage limitations, meaning the number of countries included per region varies across years.

Graph 4

```
In [94]: build_pearson_by_year(monthly)
```

```
=====
PEARSON r BY YEAR
=====
```

Global South	2015	r=0.465	p=4.5650e-04	n= 53	***
Global South	2016	r=0.530	p=6.8302e-06	n= 64	***
Global South	2017	r=0.619	p=3.0374e-08	n= 66	***
Global South	2018	r=0.321	p=8.7527e-04	n=104	***
Global South	2019	r=0.199	p=4.3965e-02	n=103	*
Global South	2020	r=0.100	p=3.0613e-01	n=106	
Global South	2021	r=0.529	p=4.2954e-10	n=121	***
Global South	2022	r=0.142	p=1.4339e-01	n=108	
Global South	2023	r=0.150	p=1.0713e-01	n=116	
Global South	2024	r=0.202	p=2.8979e-02	n=117	*
Global North	2018	r=0.098	p=6.4824e-01	n= 24	
Global North	2019	r=0.278	p=2.0961e-01	n= 22	
Global North	2020	r=0.370	p=4.6181e-03	n= 57	**
Global North	2021	r=0.240	p=4.8618e-02	n= 68	*
Global North	2022	r=0.415	p=7.8861e-04	n= 62	***
Global North	2023	r=0.222	p=8.5083e-02	n= 61	
Global North	2024	r=0.133	p=3.0976e-01	n= 60	



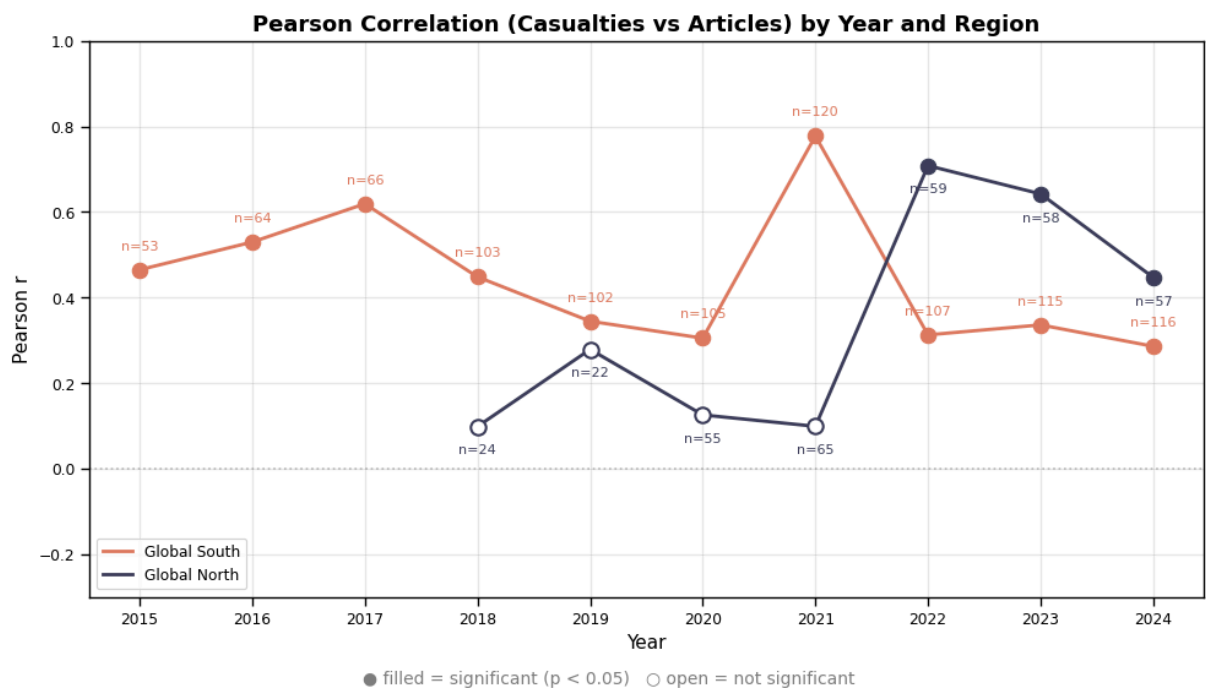
The graph shows that the Guardian is consistently less responsive to casualty spikes in the Global South over the study period, with the only notable exception being a spike in 2021. In the Global North, responsiveness remains relatively stable, with the exception of 2020 and 2022, the latter coinciding with the start of the Russia-Ukraine war.

However, as noted earlier, the data contains significant outliers: China in the Global South, and the USA, UK, and Australia in the Global North all have high article counts relative to their casualties, driven by geopolitical significance or the Guardian's core readership base. These outliers may also violate the normality assumptions underlying Pearson correlation, potentially affecting the reliability of the coefficients.

```
In [95]: EXCLUDE_GS = ['China']
EXCLUDE_GN = ['United Kingdom', 'United States', 'Australia']
monthly_no_outliers = monthly[~(
    ((monthly['region_group'] == 'Global South') & (monthly['country'].isin
    ((monthly['region_group'] == 'Global North') & (monthly['country'].isin
    ))
    build_pearson_by_year(monthly_no_outliers)
```

PEARSON r BY YEAR

Global South	2015	r=0.465	p=4.5650e-04	n= 53	***
Global South	2016	r=0.530	p=6.8302e-06	n= 64	***
Global South	2017	r=0.619	p=3.0374e-08	n= 66	***
Global South	2018	r=0.448	p=2.0861e-06	n=103	***
Global South	2019	r=0.344	p=3.9470e-04	n=102	***
Global South	2020	r=0.305	p=1.5729e-03	n=105	**
Global South	2021	r=0.777	p=1.7058e-25	n=120	***
Global South	2022	r=0.312	p=1.0525e-03	n=107	**
Global South	2023	r=0.336	p=2.4469e-04	n=115	***
Global South	2024	r=0.286	p=1.8846e-03	n=116	**
Global North	2018	r=0.098	p=6.4824e-01	n= 24	
Global North	2019	r=0.278	p=2.0961e-01	n= 22	
Global North	2020	r=0.125	p=3.6227e-01	n= 55	
Global North	2021	r=0.099	p=4.3491e-01	n= 65	
Global North	2022	r=0.708	p=3.5249e-10	n= 59	***
Global North	2023	r=0.642	p=5.5769e-08	n= 58	***
Global North	2024	r=0.447	p=4.9432e-04	n= 57	***



Excluding these outliers produces notably different results. In the Global South, all Pearson correlations become statistically significant, and in the Global North, correlations become statistically significant from 2022 to 2024. The coefficients also increase across both regions compared to when outliers were included, suggesting that

the Guardian is genuinely responsive to casualty spikes. However, that this signal is masked by countries where coverage is driven by readership demographics or geopolitical influence rather than conflict severity.

5. Conclusion

In conclusion, this project reveals three key findings about Guardian coverage of global conflicts between 2015 and 2024. First, as expected, Global South countries receive significantly fewer articles than Global North countries, confirming the proximity bias. Second, and more surprisingly, Global South articles actually have a higher median word count, suggesting the disparity is one of frequency rather than depth. Third, and most importantly, the relationship between casualties and coverage differs markedly between regions: for Global North countries the correlation is weak and statistically insignificant, while for Global South countries it is stronger and statistically significant, meaning Guardian coverage of the Global South is more directly driven by conflict severity. We hypothesise that this is because Global North coverage is inflated by countries like the USA, UK, and Australia, which are covered heavily regardless of casualties as they represent the Guardian's core readership. This is supported by our sensitivity analysis, which shows that removing those three countries strengthens the Global North correlation considerably. That said, there are some limitations worth noting that may hinder the findings of this project:

1. Severity metric

- We use fatalities as a simplified conflict severity metric. This doesn't capture displacement, conflict duration, or humanitarian scale, meaning some devastating crises are invisible to our metric.
- We did not independently verify ACLED's casualty figures. Reported fatalities in active conflict zones are inherently uncertain and may undercount deaths in areas with limited media access.

2. Statistical limitations

- The yearly Pearson correlation analysis operates on relatively small samples per year, which limits statistical power and may not fully meet normality assumptions.
- Some countries have gaps in ACLED coverage, particularly in earlier years, which could introduce bias into our correlation estimates.

3. ANOVA limitation

- We categorize countries based on levels of fatality, however some of those levels only have a few Global North countries. This small sample size in turn affects the reliability of ANOVA.

In the future, it may be worthwhile to extend this analysis to other major newspapers such as the BBC or New York Times to test whether proximity bias is universal, and to incorporate displacement data as a second severity metric for a more complete picture.